RoboCommons: Open, Evolving Evaluation for Generalist Robot Policies

Yi Ru Wang[1,2†], Carter Ung[1,2,3†], Evan Gubarev[1,2], Christopher Tan[1], Max Argus[2], Florian Shkurti[2,4], Homanga Bharadhwaj[3], Siddhartha Srinivasa[*1], Dieter Fox[*1,2]

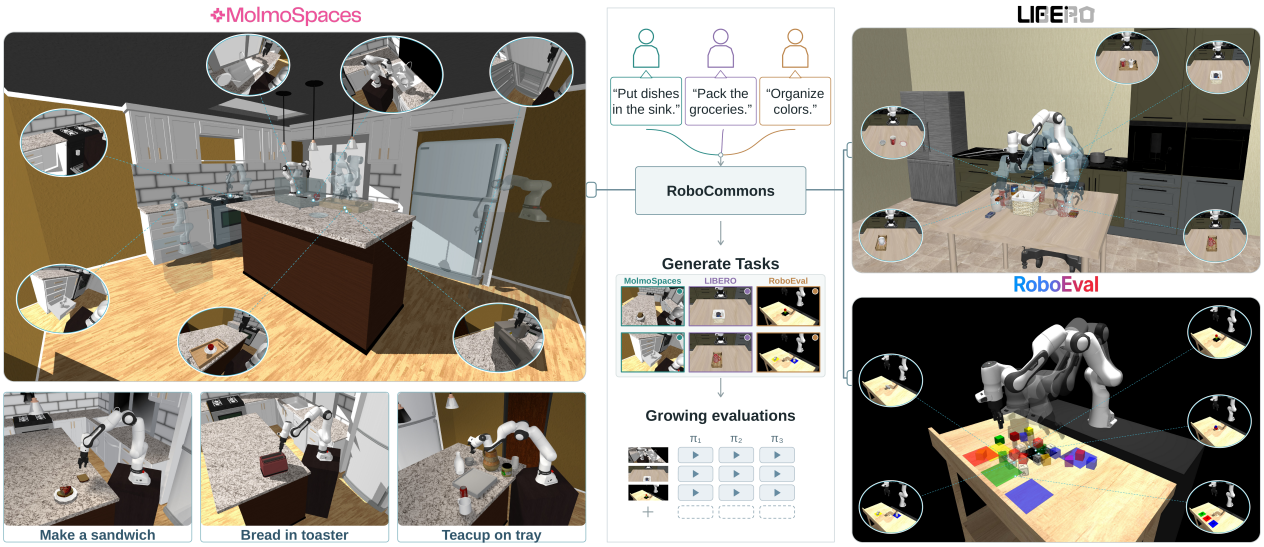

Fig. 1. Open-ended evaluation that grows alongside policy capabilities. RoboCommons translates natural-language descriptions of behaviors users wish to evaluate (*center*) into executable tasks within existing manipulation frameworks: MolmoSpaces (*left*), LIBERO (*top right*), and RoboEval (*bottom right*). Callouts show generated tasks, with three named examples below MolmoSpaces. User contributions accumulate in a shared, searchable repository, enabling policies π_i to be evaluated on targeted task subsets that probe particular capabilities and failure modes. Evaluation can thus expand beyond the behaviors anticipated at benchmark design time.

***Abstract*— Generalist robotic manipulation policies are rapidly expanding in capability, but their evaluation remains largely tied to fixed benchmarks designed and defined at benchmark design time. We introduce RoboCommons, a language-driven multi-framework harness for *open-ended evaluation* that lets users specify what they want to test while handling framework-specific task implementation. RoboCommons generates executable, verified tasks across existing manipulation frameworks and accumulates them into a reusable evaluation repository. Using more than 300 generated tasks, we show that evaluation diversity continues to grow as the repository expands. Its automated verifier produces tasks that are 98.5% correct under human adjudication while retaining 93.6% of valid tasks. Across three frameworks, RoboCommons is rated significantly more usable and less cognitively demanding compared to GenSim and Cursor in a 26-participant user study. Finally, across 15 policy–framework combinations and 42,576 episodes, targeted task families reveal distinct capability gaps that are obscured by aggregate success rates. These results show that robotic evaluation can be made easier to author, reliable to execute, and extensible as policy capabilities evolve. Webpage: `https://helen9975.github.io/RoboCommons-web/`**

†Equal contribution. *Equal advising.
[1]University of Washington, Seattle, WA, USA `yiruwang@cs.washington.edu`
[2]Allen Institute for AI, Seattle, WA, USA
[3]Johns Hopkins University, Baltimore, MD, USA
[4]University of Toronto, Toronto, ON, Canada

I. Introduction

Robot learning is rapidly shifting from specialized policies designed for narrowly defined tasks toward increasingly generalist models capable of following diverse instructions, manipulating many objects, and operating across a growing range of environments. As the capabilities of these policies expand, however, the way we evaluate them has remained comparatively static. Today's robotic manipulation benchmarks typically consist of fixed collections of expert-designed tasks that probe a predetermined set of behaviors [1], [2], [3], [4], [5], [6]. These benchmarks have been critical for measuring progress, but a finite task suite can only test the capabilities and failure modes anticipated by its designers. As policies become more capable, the space of relevant behaviors grows along many dimensions, including object types, spatial configurations, semantic instructions, physical interactions, and compositions of skills. A model may therefore improve substantially or fail in important and unexpected ways, without either change being reflected by a fixed set of evaluation scenarios. This creates a growing mismatch: *while policy capabilities can evolve continuously, the evaluations used to characterize them are largely fixed at benchmark design time*.

Other areas of machine learning have responded to similar challenges by making evaluation more dynamic.

Dynabench [7] continually introduces human-authored examples that expose weaknesses of contemporary models, LiveBench [8] periodically refreshes its evaluation set as models improve, and Procgen [9] evaluates agents over procedurally generated environment distributions rather than a small number of fixed instances. These efforts share a common principle: *evaluation should be able to evolve alongside the systems it measures*. We seek to bring this principle to robotic manipulation through *open-ended evaluation*, where new task semantics and scenarios can be continually authored rather than fully enumerated at benchmark design time.

Unlike a language benchmark, a manipulation task must be executable: assets and initial states must be configured, physical interactions must remain stable, success conditions must be implemented, and the resulting environment must interface correctly with the policy. Moreover, these abstractions differ across manipulation frameworks, each with its own task APIs, assets, simulator conventions, and evaluation protocols. Consequently, turning a newly observed capability or failure mode into a reproducible test often requires substantial expertise in the underlying simulation stack. Recent systems such as GenSim [10] and RoboGen [11] demonstrate that foundation models can automate parts of robotic task construction, primarily to expand training tasks and supervision. We instead ask a complementary question: *can generative task construction make evaluation itself continuously extensible*? Doing so requires treating generated tasks not merely as transient sources of training experience but as reproducible evaluation artifacts that can be executed across policies, accumulated over time, and selectively reused to interrogate particular capabilities.

To this end, we introduce **RoboCommons**, a language-driven task orchestration harness for open-ended evaluation of robotic manipulation policies. As shown in Fig. 1, RoboCommons allows users to express behaviors they wish to evaluate through natural language and orchestrates their realization as executable evaluation tasks within existing manipulation frameworks. Rather than replacing the rich simulation infrastructure already developed by the community, RoboCommons builds on top of it, providing a common authoring workflow while delegating execution to framework-specific backends. This design allows RoboCommons to operate across heterogeneous evaluation ecosystems and to incorporate additional frameworks as the robotics landscape evolves. Crucially, generated tasks do not disappear after a single evaluation. RoboCommons accumulates them into a growing task repository, where evaluations can be searched and retrieved using natural-language descriptions and keywords. Users can therefore construct targeted subsets of tasks corresponding to behaviors of interest, execute policies on those subsets, and expose capability-specific strengths and weaknesses. Evaluation becomes not only a mechanism for producing an aggregate score but also a tool for actively *interrogating* policy behavior.

We apply RoboCommons across three robotic manipulation frameworks and use it to generate a repository of more than 300 executable evaluation tasks. We study how the resulting task space evolves as additional tasks are introduced and find that increasing the size of the repository continues to expand the diversity of behaviors represented in the evaluation rather than simply producing additional instances of a small set of existing scenarios. Finally, we evaluate five state-of-the-art manipulation policies across the generated tasks. The resulting evaluations reveal substantial differences in policy behavior across tasks and targeted task subsets, exposing capability gaps that can be obscured when performance is summarized only through conventional aggregate benchmark scores. Together, these results demonstrate how lowering the cost of task construction can transform evaluation from a fixed suite that policies repeatedly optimize against into an evolving process that grows alongside the capabilities it is intended to measure.

Our contributions are threefold:

- **Open-ended evaluation orchestration.** We introduce RoboCommons, a language-driven harness that separates evaluation intent from framework-specific task implementation and validate it across three manipulation frameworks.
- **A growing evaluation repository.** We generate 300+ executable manipulation tasks and show that increasing repository scale continues to expand evaluation diversity.
- **Capability-oriented evaluation.** We evaluate five state-of-the-art manipulation policies and show that targeted task subsets expose distinct gaps in policy performance that can be obscured by aggregate benchmark scores.

II. Related Work

A. Evaluation and Benchmarking for Robotic Manipulation

Robotic manipulation systems are most commonly evaluated using fixed benchmark suites composed of predefined task instances, environments, and success criteria, such as RLBench [1], LIBERO [2], RoboCasa [3], Behaviour-1k [12], ManiSkill [4], Colosseum [5], Simpler [13], and RoboEval [6]. While these benchmarks have been instrumental in standardizing evaluation, they define evaluation over a fixed and finite set of expert-authored tasks, with task structure, constraints, and success criteria encoded procedurally and not exposed for user modification. Although some benchmarks include natural language annotations or language-conditioned tasks, language is typically treated as documentation or policy input rather than as part of the executable task specification, making it difficult to introduce task variations or alternative notions of success without modifying benchmark code. Recent work has explored complementary directions for scaling evaluation: RoboArena [14] democratizes who evaluates and where evaluation occurs through crowd-sourced, double-blind pairwise comparisons over unconstrained real-world tasks, while Polaris [15] improves the fidelity and scalability of simulation-based evaluation via real-to-sim scene reconstruction, but retains fixed, expert-authored tasks and success criteria. In contrast, our work focuses on democratizing what is evaluated by enabling users to author, modify, and refine executable task specifications through language, while preserving structure, reproducibility, and comparability.

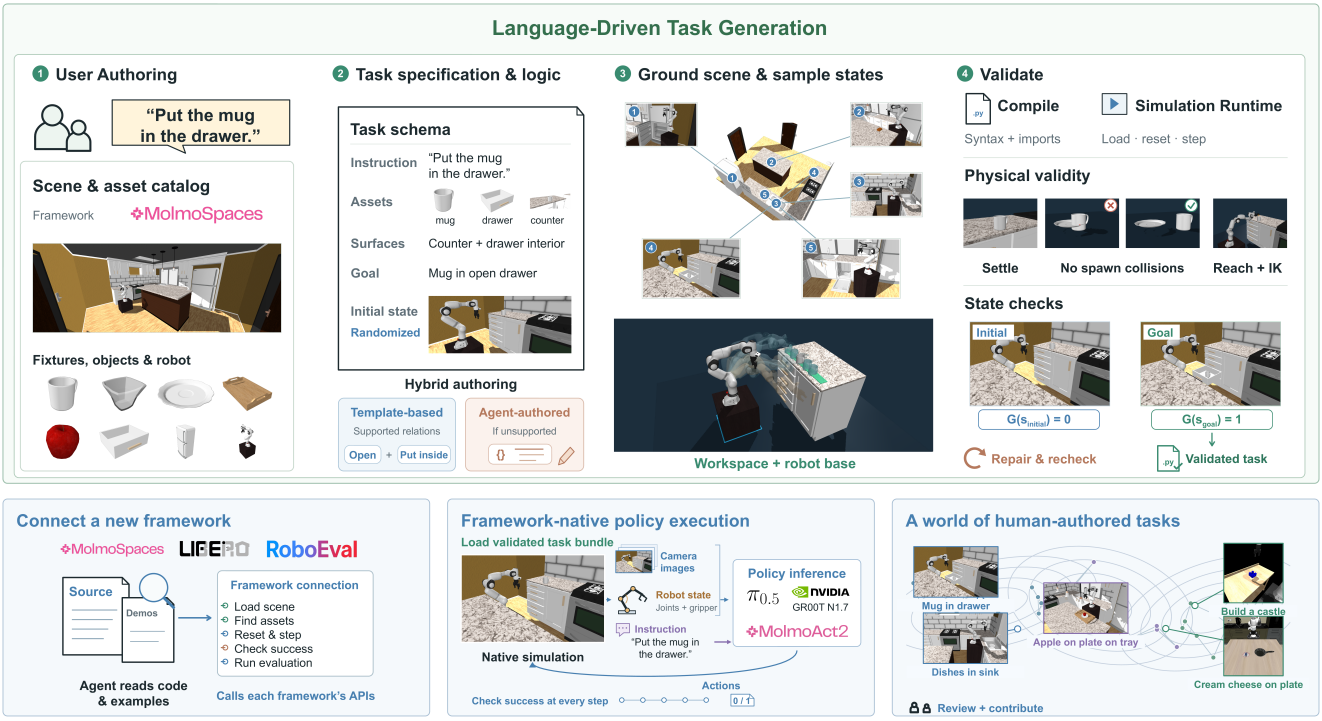

Fig. 2. System overview. The system converts a natural-language behavior description into a validated, framework-native evaluation task. (*Top*) A user specifies a behavior and selects a framework, scene, and assets. A VLM agent defines the instruction, relevant assets and surfaces, goal condition, and randomized initial state, using templates for supported relations and authoring new logic when needed. It then grounds the task in the scene, samples states within the robot's workspace, and validates compilation, simulator execution, stable and collision-free placement, inverse-kinematics reachability, and the goal condition, requiring $G(s_{\text{initial}}) = 0$ and $G(s_{\text{goal}}) = 1$. Failed checks trigger repair and revalidation. (*Bottom left*) To connect a new framework, the agent reads its code and examples and exposes native operations for scene loading, asset discovery, simulation, success checking, and evaluation. (*Bottom center*) Validated task bundles run in the framework's native simulator, which provides observations and instructions to the evaluated policy, executes its actions, and checks success at each step. (*Bottom right*) User requests accumulate into a growing bank of validated, human-specified tasks that can be reviewed, run, extended, and shared.

B. LLM-Based Task and Environment Generation

Recent work has explored using large language models to generate robotic tasks, environments, rewards, or curricula at scale. Systems such as GenSim [10], Gen2Sim [16], RoboGen [11], and AnyTask [17] leverage LLMs to synthesize tasks or simulation assets, while Eureka [18] and Eurekaverse [19] use language models to automatically generate reward functions or learning curricula. These approaches primarily target data generation and training diversity, rather than evaluation itself: generated tasks are treated as inputs to learning pipelines, and the task space is not exposed to users as a controllable or interpretable evaluation interface. Task generation is typically decoupled from mechanisms for enforcing reproducibility, tracking task lineage, or systematically relating task variations to evaluation outcomes. Language has also been used to guide robot behavior at execution time, as in SayCan [20] and Code as Policies [21], where it serves as a high-level planning or control signal while task definitions and success criteria remain fixed and externally specified. In contrast, our work treats language as a first-class interface for evaluation: natural language instructions are compiled into structured, executable task specifications with explicit asset definitions, initialization distributions, and success predicates, enabling users to author reproducible families of semantically related evaluation tasks that support controlled variation and systematic comparison.

III. Methods

In this section, we will describe the task generation system and show the extensibility of the language-driven agent harness to MuJoCo frameworks. We explain how the system captures human intent and uses a combination of LLM agents and native framework infrastructure to translate and fully validate this into task-specific logic as an executable artifact.

A. Domain and Task Representations

Our framework captures human intent by treating natural language as an executable interface for task specification. As a preliminary to authoring new tasks, a domain specifies a structured, controllable environment to author tasks:

$$D = \big(M, P, \mathcal{F}, \mathcal{A}, K\big), \tag{1}$$

where M is the simulation framework, P is its preset scene, $\mathcal{F} = \{f_1, \ldots, f_N\}$ is the asset inventory, $\mathcal{A}$ is the catalog of available object types, whether statically fixed or spawnable, and K is the chosen robot. These variables are bundled together as a configuration file to define a family of candidate tasks $\mathcal{T}(D)$.

Given a language request, the agent constructs one task $t \in \mathcal{T}(D)$ as

$$t = \big(\mathcal{F}_t, \mathcal{A}_t, \rho_0, G, \ell, \mathcal{W}_t\big), \tag{2}$$

where $\mathcal{F}_t \subseteq \mathcal{F}$ selects task-relevant fixtures (static assets), $\mathcal{A}_t$ is a multiset of object instances drawn from the catalog, ρ_0 is the initial-state distribution induced by a seeded reset procedure, $G : \mathcal{S} \to \{0, 1\}$ evaluates success conditions, and ℓ is the instruction fed to the policy at test time.

The placement workspace $\mathcal{W}_t$ collects the surface regions required to transition from an initial state to the goal state while maintaining feasible reach for the robot, such as transitioning a bowl from the counter to the adjacent sink. For a horizontal support surface f, its placement region is

$$\mathcal{W}_f = \big\{(x, y, h(f)) \in \mathbb{R}^3 \mid (x, y) \in B_f\big\}, \tag{3}$$

where B_f is the usable horizontal planar region and $h(f)$ is the fixture's surface (support) height measured from the instantiated scene. Placements for assets that have interior cavities also use measured regions inside the selected asset. These regions specify placement locations, considering for object size, orientation, and robot reachability.

B. Task Authoring

Agent Scaffolding. Given a task t as defined in III-A, the task is fed into a Vision-Language Model (VLM) agent packaged with skills and a multi-tool interface for orchestrating privileged access in simulation. The skills directory acts as a scaffold to help guide the agent during open-ended exploration inside the hosted simulation framework. A primary skill document orchestrates a structured workflow, strategies for tool and subagent execution, and self-feedback loops while still preserving general reasoning. Subagents with their own individual skills build out different facets of the task, including discovering framework context and building out a framework adapter to represent native authoring patterns, creating task-specific logic to define the goal and failure definitions, sampling feasible robot-workspace regions to spawn the assets in the initial and goal states, and executable validation. The tools are made general-purpose to probe inside simulation and retrieve data in episodes, e.g., retrieving asset properties (pose, bounding boxes, joints, sites), robot state, and settling physics.

Hybrid-Authoring. We design a hybrid authoring strategy gated between template-based and LLM-based code generation. Modular template-based approaches can effectively model tasks with simple relational predicates (On, Beside, Rotate) when the task supports them. These can model the task logic by deterministically measuring scene properties like object bounding boxes and workspace footprints to apply spatial relations so that the task goal can be defined without invoking LLM tokens and relying on an agent to author the task logic. However, complex tasks that cannot be modeled by available relational predicates–the target letter is on the top face of the cube–require the agent to author task logic, where the task authoring process benefits from the native framework APIs that the agent autonomously picks up. The VLM agent gates these two strategies to leverage functional consistency, bridging deterministic templates and LLM coding agents for broader coverage.

Scene Construction. Given the created task definition, the objective is for the VLM agent to load the selected scene P and sample a measured workspace $\mathcal{W}_f$ compatible with the robot base K. The selected strategy is left open to the VLM agent whether to use native framework methods or general shared simulation APIs built into the simulation pipeline. For instance, in MolmoSpaces, the framework provides its own checks for reachability with inverse kinematics implementation, samples robot positions based on an occupancy map, and samples object placements with workspace measurements to validate candidates when available; the skills encourage the agent to leverage native tools for consistent authoring in the framework.

C. Logical Validation

Language-defined tasks are only meaningful if they are both executable and physically realizable. Each synthesized task implementation is therefore subjected to a multi-stage validation pipeline before being admitted as a task artifact.

Code Syntax. The natural output of the agent is represented through Python-based simulation code in MuJoCo. Therefore, the natural first objective of validation is to enforce mechanical software checks independent of the requested task instruction. The generated task code is subjected to static analysis to detect syntactic errors and forbidden patterns, compiled in an isolated execution environment to detect import and definition errors, and instantiated in simulation to detect runtime failures while stepping physics in the episode.

Physical constraints. After code checks are done to validate that the output can be stepped in simulation, we prove the initial state proposed is physically valid and compliant with our quality factors for evaluation. This includes ensuring episode resets provided by testing seeded distributions of asset poses in the initial state, tabletop objects are stable and settled onto the desired surfaces, assets do not initially collide when spawned into the scene, and lastly, all spawnable assets within the chosen workspace region are reachable from the robot through inverse kinematics.

Goal-state verification. To ensure that the defined task success is physically plausible, the agent attempts to build a goal configuration where its objective optimizes to detect success directly in a simulation episode. The agent spawns a test scene with robot actions frozen and builds a goal configuration by sampling placement candidates and simulating timesteps forward to allow contacts to settle. The success predicate G must evaluate to true after settling and remain true over an extended horizon, ensuring that the goal configuration is both achievable and stable under the simulator's physics model.

Iterative repair. Self-feedback loops are embedded throughout the authoring pipeline. The skills that scaffold the agent's reasoning provides guidance to diagnose and make targeted corrections using the general set of tools probing simulation, state such as assets' geometric properties, joints,

TABLE I. User study results (N=26). Mean ± 95% CI margin. SUS [22]: 0–100 scale. TLX: unweighted mean of five NASA-TLX [23] subscales, normalized to 0–100. Rank: 1=best. Preferred: share selecting each system overall.

System	SUS ↑	TLX workload ↓	Usability rank ↓	Preferred (%)
GenSim [10]	52.5±9.3	41.8±9.0	2.7±0.2	8
Cursor [24]	68.8±7.8	36.7±10.4	2.0±0.2	23
RoboCommons	**83.4±6.9**	**18.6±7.7**	**1.3±0.3**	**69**

contacts, and poses. Repairs may adjust object placements, relax geometric constraints, or rewrite components of the success predicate, depending on the nature of the failure fed back to the agent. The validation pipeline is then reran every time the task code is modified until all checks pass, guaranteeing that tasks satisfy executability and physical consistency.

IV. Experiments and Results

The value of language-driven evaluation depends on whether users can turn new evaluation questions into correctly implemented tasks that reveal policy strengths and limitations. We first examine whether RoboCommons makes task authoring more usable and less demanding than existing interfaces (Sec. IV-A). Because implementation errors can confound policy assessment, we then evaluate whether automated verification filters out incorrect tasks while retaining valid candidates (Sec. IV-B). We next investigate whether expanding the repository broadens evaluation coverage across language, visual observations, and motion (Sec. IV-C). Building on this task set, we examine how targeted task families and linked variations expose performance differences and failure patterns obscured by aggregate scores (Sec. IV-D). Finally, we assess whether relative task difficulty is consistent between simulation and hardware (Sec. IV-E).

A. Accessibility of Task Authoring

Experimental setting. We compare RoboCommons with GenSim [10] and Cursor [24] in a within-subjects user study ($N = 26$). Each participant uses all three systems to author the same constrained block-building task. We measure perceived usability with the System Usability Scale (SUS) [22], workload with a five-subscale composite adapted from NASA-TLX [23], and preference through rankings and a forced-choice selection. The workload composite averages Mental Demand, Temporal Demand, Effort, Frustration, and reversed Performance, normalized to 0–100 with lower scores indicating lower workload. Table I summarizes the results; intervals denote 95% confidence interval margins.

The authoring workflow improves perceived usability and reduces workload. RoboCommons achieves a SUS score of 83.4 ± 6.9, compared with 52.5 ± 9.3 for GenSim and 68.8 ± 7.8 for Cursor. It also yields lower perceived workload: 18.6 ± 7.7, versus 41.8 ± 9.0 and 36.7 ± 10.4, respectively. Paired Wilcoxon signed-rank tests support both the usability advantage (GenSim: $p < 0.001$; Cursor: $p = 0.0017$) and the workload reduction (GenSim: $p = 0.0007$; Cursor: $p = 0.0019$). Preference follows the same pattern: 69% of participants choose RoboCommons, compared with 23% for Cursor and 8% for GenSim. These results support the accessibility of the workflow for the studied authoring task.

B. Reliability of Generated Evaluation Tasks

Experimental Setting. We evaluate the automated verification pipeline against human verification on 397 generated tasks, sampling tasks that both pass and fail the verifier. Each task receives a pass/fail decision from the automated code and visual review stages and is independently labeled by a human as correct or incorrect, based on whether the generated initialization, assets, and success predicate realize the intended task.

Automated verification substantially improves the reliability of the resulting task set. Fig. 3 compares automated verification against human verification. Among the 343 tasks that pass verification, 338 (98.5%) are judged correct by human reviewers, indicating that very few faulty tasks remain after filtering. At the same time, the verifier retains 93.6% of human-judged correct tasks, showing that this reliability does not require discarding most usable candidates. Overall, verification produces a task set that is highly consistent with human judgement while preserving the large majority of valid tasks.

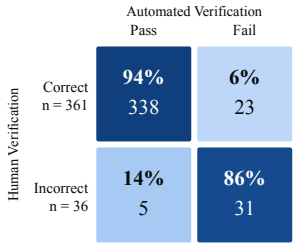

Fig. 3. Automated verification against human adjudication. Rows are the human verification, columns the automated one. Percentages are normalized within each row ($n = 361$ correct, $n = 36$ incorrect); the count in each cell is the number of tasks.

Verification errors are conservative, favoring false rejection over false acceptance. The two error types have asymmetric costs: accepting an incorrect task introduces a faulty task into evaluation, whereas rejecting a correct task primarily reduces throughput. The verifier exhibits this preferable failure mode: only 5 incorrect tasks pass verification, compared with 23 correct tasks that are rejected. Accordingly, while 42.6% of rejected tasks are actually usable, only 1.5% of tasks that pass verification are incorrect. This asymmetry prioritizes the reliability of the benchmark over retaining every valid generated task.

C. Growth and Diversity of Evaluation Coverage

Experimental setting. To determine whether additional tasks broaden evaluation coverage, we analyze 300 tasks generated using three domains: blocks, kitchen, and LIBERO scenes, each contributing exactly 100 tasks. We measure diversity using the Vendi score [25], an effective count of distinct samples derived from pairwise similarities, and compare subsets drawn from one, two, or three domains at fixed task counts (Fig. 4, top). We examine four aspects: language instruction, scene and goal appearance, goal-relative visual change, and robot joint trajectory. Instructions are embedded with Qwen3-Embedding-8B [26], and rendered initial and goal images with DINOv2 [27] (ViT-S/14); appearance is the concatenation of the initial and goal embeddings, and goal-relative change their normalized difference, which distinguishes tasks that share a scene but ask for different outcomes. These three aspects use

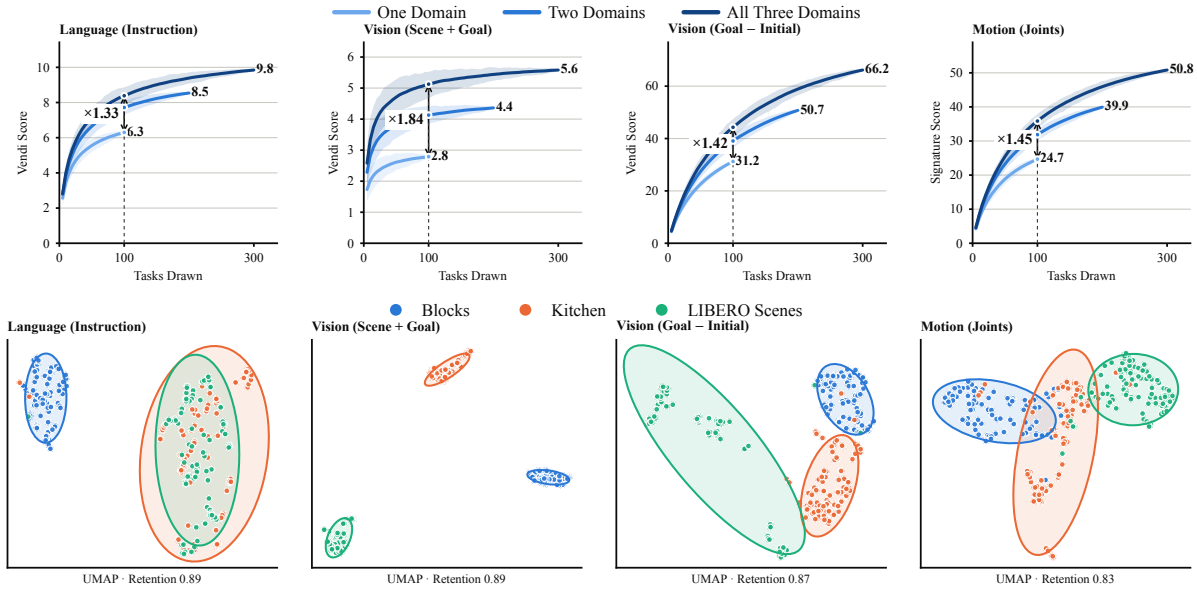

Fig. 4. Task diversity increases as the source pool broadens. Columns measure four aspects of task diversity: language instruction, scene and goal appearance, goal-relative visual change, and joint motion. **Top:** diversity when sampling from one, two, or all three domains; shaded regions show 95% confidence intervals across draws. Ratios annotate the increase from a single-domain to a three-domain pool at matched task budgets. **Bottom:** UMAP projections of the same tasks, colored by domain, with contours containing 80% of each domain's samples. Visual representations separate strongly by domain, motion separates partially, while language shows substantial overlap between MolmoSpaces kitchen and LIBERO tasks. All diversity scores are computed in the original representation space. Motion uses a signature kernel over joint trajectories and is therefore shown on its own score scale.

cosine similarity between embeddings, while motion uses a signature kernel over joint trajectories [28].

Source breadth expands coverage at a fixed task budget. At a fixed budget of 100 tasks, the three-domain pool yields 1.33×, 1.84×, 1.42×, and 1.45× the diversity of the single-domain reference in language, scene and goal appearance, goal-relative visual change, and motion, respectively. While within a domain, diversity grows as more tasks are added, broadening the source pool further increases the diversity at comparable task counts. This supports the value of accessing multiple task sources through a common authoring workflow: repository size and source breadth make different contributions to measured coverage.

Instruction diversity captures only part of the expanded coverage. The relative gain from combining domains is smallest for language and largest for scene and goal appearance, with substantial gains also present in goal-relative visual change and motion. The projections in Fig. 4 (bottom) qualitatively echo these differences: visual representations separate strongly by domain, language representations overlap more, and motion exhibits both separation and overlap. All diversity scores are computed in the original representation spaces. These measurements support evaluating repository breadth beyond instruction similarity, although domain-specific appearance and robot embodiment may also contribute to the separation.

D. Diagnostic Value for Evaluating Generalist Policies

Experimental setting. We examine whether the task repository supports targeted comparisons of policy capabilities and failure patterns. We organize tasks into five families: horizon, semantic grounding, articulation and affordance, clutter, and spatial relations. The evaluation spans RoboEval Blocks, MolmoSpaces Kitchen, and LIBERO, comprising 15 policy–framework combinations and 42,576 episodes, with 100 tasks per combination. We evaluate MolmoAct2 [29], MolmoBot [30], $\pi_{0.5}$ [31], GR00T N1.7 [32], Cosmos [33], X-VLA [34], VLA-JEPA [35], and FastWAM [36]. Table II reports family-level performance, while Fig. 5 compares

TABLE II. Diagnostic performance across user-specified hypothesis clusters. Tasks are grouped into horizon, semantic grounding, articulation and affordance, clutter, and spatial relations. Each cell reports episode-weighted success rate (%) ± binomial standard error. **Overall** is computed over all evaluated episodes rather than averaged across clusters. Results cover 15 policy–framework combinations and 42,576 episodes over the finalized 300-task set, with every policy evaluated on all 100 specs of each framework it was run in. Darker shading indicates higher success.

Method	Horizon	Semantic grounding	Articulation & affordance	Clutter	Spatial relations	Overall
RoboEval – Blocks						
MolmoAct2 [29]	8.8 ±1.6	0.9 ±0.7	16.9 ±2.1	66.4 ±2.3	13.7 ±1.7	24.8 ±1.1
MolmoBot [30]	0.3 ±0.3	0.5 ±0.5	10.9 ±1.7	10.2 ±1.5	0.8 ±0.4	5.0 ±0.5
$\pi_{0.5}$ [31]	0.0 ±0.0	0.0 ±0.0	0.3 ±0.3	0.4 ±0.3	0.0 ±0.0	0.2 ±0.1
GR00T N1.7 [32]	0.9 ±0.5	0.9 ±0.7	8.8 ±1.6	15.5 ±1.8	2.8 ±0.8	6.5 ±0.6
Cosmos [33]	0.0 ±0.0	1.0 ±0.7	2.7 ±1.0	13.6 ±1.9	2.6 ±0.9	4.3 ±0.5
MolmoSpaces – Kitchen						
MolmoAct2 [29]	0.0 ±0.0	0.0 ±0.0	4.6 ±1.4	4.5 ±1.5	3.5 ±1.3	2.6 ±0.5
MolmoBot [30]	9.0 ±2.0	0.0 ±0.0	11.4 ±2.1	17.5 ±2.7	4.0 ±1.4	8.5 ±0.9
$\pi_{0.5}$ [31]	0.0 ±0.0	1.5 ±0.9	4.2 ±1.3	0.0 ±0.0	1.0 ±0.7	1.4 ±0.4
GR00T N1.7 [32]	0.5 ±0.5	1.0 ±0.7	5.4 ±1.5	8.0 ±1.9	2.0 ±1.0	3.5 ±0.6
Cosmos [33]	0.0 ±0.0	0.5 ±0.5	5.0 ±1.4	0.0 ±0.0	2.0 ±1.0	1.6 ±0.4
LIBERO						
MolmoAct2 [29]	26.7 ±1.4	0.0 ±0.0	9.4 ±0.9	1.9 ±0.4	8.2 ±0.8	8.5 ±0.4
$\pi_{0.5}$ [31]	17.7 ±1.2	0.0 ±0.0	6.4 ±0.7	41.7 ±1.6	12.9 ±1.0	14.2 ±0.5
X-VLA [34]	5.5 ±0.7	0.0 ±0.0	4.9 ±0.7	6.6 ±0.8	2.5 ±0.5	3.6 ±0.2
VLA-JEPA [35]	7.9 ±0.8	0.0 ±0.0	3.9 ±0.6	0.0 ±0.0	0.3 ±0.1	2.2 ±0.2
FastWAM [36]	3.9 ±0.6	0.0 ±0.0	9.6 ±0.9	0.2 ±0.1	5.1 ±0.7	3.6 ±0.2

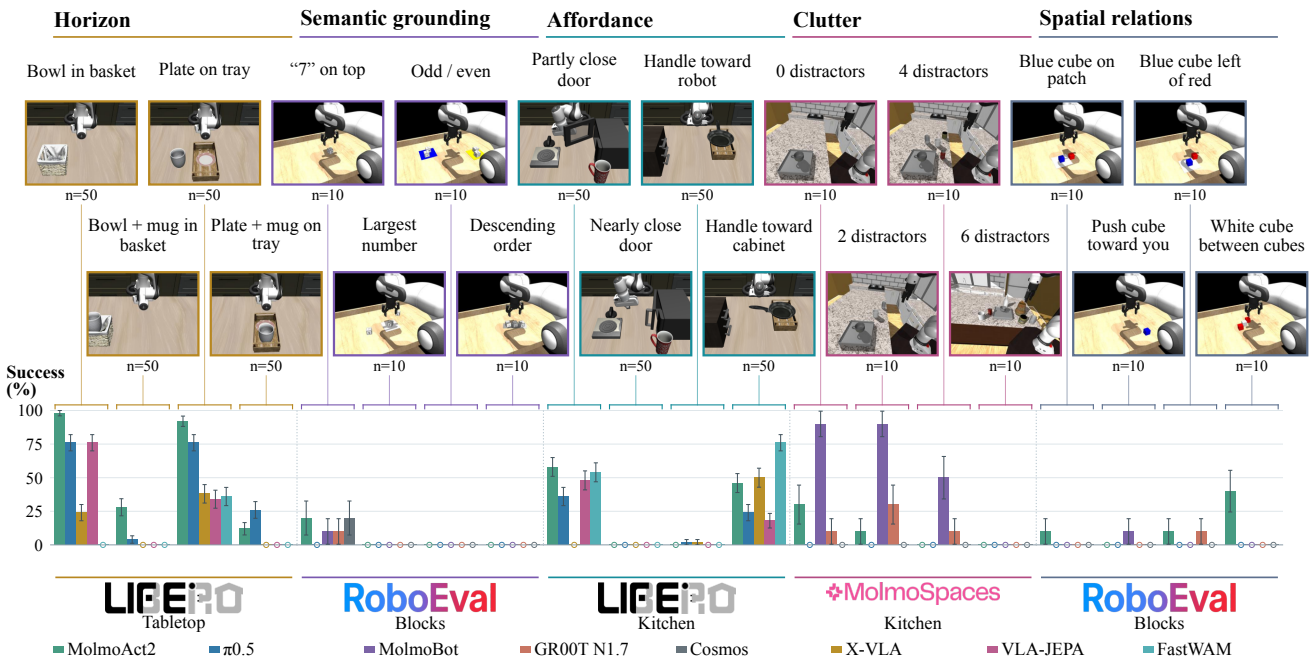

Fig. 5. Task subsets reveal policy behavior obscured by family-level performance. Each panel shows four tasks from one capability family and benchmark setting: horizon (LIBERO Tabletop), semantic grounding (RoboEval Blocks), affordance (LIBERO Kitchen), clutter (MolmoSpaces Kitchen), and spatial relations (RoboEval Blocks). Horizon compares single- and two-object placement for two object–receptacle pairs; affordance compares partial and near-complete door closure and two handle orientations; and clutter holds the teacup-on-tray goal fixed while increasing distractors. Semantic-grounding tasks sample related instructions, whereas spatial-relation tasks span distinct relation types and are not intended to represent the full family. Goal snapshots and per-policy episode counts n appear above the bar groups. Bars show success rates with ± 1 binomial standard error. ∘ denote evaluated policies with 0% success.

selected variations within each family.

Task families distinguish shared limitations from policy-specific profiles. Overall success does not exceed 24.8% in any evaluated policy–framework combination, but family-level scores reveal distinct performance profiles. On RoboEval Blocks, MolmoAct2 achieves 24.8% overall success, ranging from 0.9% on semantic-grounding tasks to 66.4% on clutter tasks. Semantic grounding is a shared limitation: no policy exceeds 1.5% in any framework, and all five policies evaluated in LIBERO achieve 0%. By contrast, clutter produces the largest between-policy spread in each framework: 66.0, 17.5, and 41.7 percentage points on Blocks, Kitchen, and LIBERO, respectively. The decomposition therefore separates families with near-universal failure from those that discriminate among policies, while not isolating the underlying causes.

Linked variations identify where performance deteriorates within a family. The task comparisons in Fig. 5 provide finer resolution than family averages. Adding a second placement reduces average success from 56.4% to 7.6% on the tray task and from 54.8% to 6.4% on the basket task, exposing difficulty with these multi-placement horizon variants. Affordance tasks show a different sensitivity: four policies achieve 36–58% success for partial microwave closure, while every policy fails every episode of near-complete closure. In the clutter sequence, MolmoBot achieves 90% with two distractors, 50% with four, and 0% with six. Spatial tasks are near-zero throughout, with pooled success between 0.7% and 8.0% across the four goals; the single exception is MolmoAct2 at 40% on *between*, which no other policy solves at all. Semantic-grounding variants are less discriminative: occasional success at digit identification and near-universal failure on comparison, parity, and ordering tasks. Together, these comparisons locate performance drops associated with additional placements, stricter completion requirements, distractors, and relational goals, while also identifying tasks whose near-zero success limits finer diagnosis.

Policy rankings depend on the evaluation setting. MolmoAct2 leads RoboEval Blocks at 24.8% overall success but reaches 2.6% on MolmoSpaces Kitchen, where MolmoBot leads at 8.5%. Because tasks, environments, and control interfaces differ across frameworks, these changes cannot be attributed to any one factor. They show why conclusions about policy performance should be tied to the evaluated setting, and motivate using the shared workflow to investigate behavior across multiple settings.

E. Simulation-to-Real Validation

Experimental setting. We investigate whether the relative difficulty of simulation-authored tasks is informative on hardware. We reconstruct a real tabletop scene in RoboCommons, matching its object set, layout, and camera pose (Fig. 6, left), and evaluate MolmoAct2 and $\pi_{0.5}$ on six tasks in both simulation and the real world. Figure 6 (right) compares task difficulty rankings, with rank 1 denoting the easiest task.

Relative task difficulty shows agreement across simulation and hardware. MolmoAct2 exhibits strong rank agreement (Spearman $\rho = 0.93$, $p = 0.008$). $\pi_{0.5}$ shows a positive but less conclusive association ($\rho = 0.79$,

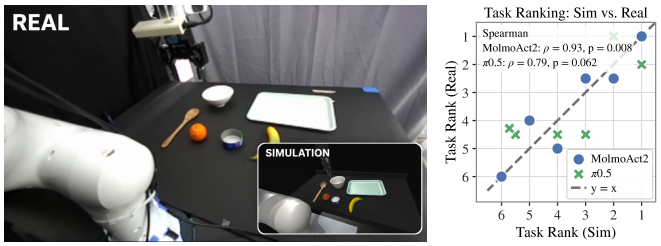

Fig. 6. Task difficulty ordering transfers from simulation to hardware. Left: a real tabletop scene and the RoboCommons scene reconstructed to match it, sharing object set, layout, and camera pose. **Right:** each task's difficulty rank in simulation against its rank on the real robot, for two policies; rank 1 is the easiest task and the dashed line is $y = x$. Ordering is preserved for MolmoAct2 (Spearman $\rho = 0.93$, $p = 0.008$) and more weakly for $\pi_{0.5}$ ($\rho = 0.79$, $p = 0.062$), whose tasks tie across the middle of the range.

$p = 0.062$), with several tasks tied in the middle of the ranking. These results provide initial evidence that relative difficulty can persist across simulation and hardware for the tested tasks, particularly for MolmoAct2, demonstrating the value and potential of leveraging simulation-based evaluations for policy assessment.

V. Conclusion and Discussion

We introduced RoboCommons, a language-driven harness for open-ended evaluation of robotic manipulation policies. By separating evaluation intent from framework-specific implementation, RoboCommons enables users to author executable tasks across existing manipulation frameworks and accumulate them into a reusable repository. Our results support the usability of this workflow and the reliability of its verification pipeline. Broadening the task sources expands measured coverage across language, visual observations, and motion, while targeted task families and linked variations expose policy strengths and limitations obscured by aggregate scores. Together, these directions offer opportunities to further expand the scope and robustness of open-ended task generation.

Limitations and Future Work. The current validation pipeline provides a foundation for ensuring the functional quality of generated tasks through automated checks. Building on this foundation, future work can extend validation to incorporate robot policies in the loop, enabling richer evaluation of semantic fidelity and temporal behavior. Integrating vision tools and part-segmentation methods may further strengthen the association between asset geometry, semantic parts, and manipulation affordances. These automated capabilities can also be complemented by post-hoc human-in-the-loop review, allowing users to assess whether generated task artifacts capture the intended language instructions.

References

[1] S. James, Z. Ma, D. R. Arrojo, and A. J. Davison, "Rlbench: The robot learning benchmark & learning environment," 2019.

[2] B. Liu, Y. Zhu, C. Gao *et al.*, "Libero: Benchmarking knowledge transfer for lifelong robot learning," 2023.

[3] S. Nasiriany, A. Maddukuri, L. Zhang *et al.*, "Robocasa: Large-scale simulation of everyday tasks for generalist robots," 2024.

[4] T. Mu, Z. Ling, F. Xiang *et al.*, "Maniskill: Generalizable manipulation skill benchmark with large-scale demonstrations," 2021.

[5] W. Pumacay, I. Singh, J. Duan, R. Krishna, J. Thomason, and D. Fox, "The colosseum: A benchmark for evaluating generalization for robotic manipulation," 2024.

[6] Y. R. Wang, C. Ung, C. Tan *et al.*, "Roboeval: Where robotic manipulation meets structured and scalable evaluation," 2025.

[7] D. Kiela, M. Bartolo, Y. Nie *et al.*, "Dynabench: Rethinking benchmarking in nlp," 2021.

[8] C. White, S. Dooley, M. Roberts *et al.*, "Livebench: A challenging, contamination-limited llm benchmark," in *International Conference on Learning Representations*, vol. 2025, 2025, pp. 91 595–91 631.

[9] K. Cobbe, C. Hesse, J. Hilton, and J. Schulman, "Leveraging procedural generation to benchmark reinforcement learning," in *International conference on machine learning*. PMLR, 2020, pp. 2048–2056.

[10] L. Wang, Y. Ling, Z. Yuan *et al.*, "Gensim: Generating robotic simulation tasks via large language models," *arXiv preprint arXiv:2310.01361*, 2023.

[11] Y. Wang, Z. Xian, F. Chen *et al.*, "Robogen: Towards unleashing infinite data for automated robot learning via generative simulation," 2024.

[12] C. Li, R. Zhang, J. Wong *et al.*, "Behavior-1k: A human-centered, embodied ai benchmark with 1,000 everyday activities and realistic simulation," 2024.

[13] X. Li, K. Hsu, J. Gu *et al.*, "Evaluating real-world robot manipulation policies in simulation," 2024.

[14] P. Atreya, K. Pertsch, T. Lee *et al.*, "Roboarena: Distributed real-world evaluation of generalist robot policies," 2025.

[15] A. Jain, M. Zhang, K. Arora *et al.*, "Polaris: Scalable real-to-sim evaluations for generalist robot policies," 2025.

[16] P. Katara, Z. Xian, and K. Fragkiadaki, "Gen2sim: Scaling up robot learning in simulation with generative models," 2023.

[17] R. Gong, X. Zhang, J. Shang *et al.*, "Anytask: an automated task and data generation framework for advancing sim-to-real policy learning," 2026.

[18] Y. J. Ma, W. Liang, G. Wang *et al.*, "Eureka: Human-level reward design via coding large language models," 2024.

[19] W. Liang, S. Wang, H.-J. Wang, O. Bastani, D. Jayaraman, and Y. J. Ma, "Eurekaverse: Environment curriculum generation via large language models," 2024.

[20] M. Ahn, A. Brohan, N. Brown *et al.*, "Do as i can, not as i say: Grounding language in robotic affordances," 2022.

[21] J. Liang, W. Huang, F. Xia *et al.*, "Code as policies: Language model programs for embodied control," 2023.

[22] J. Brooke *et al.*, "Sus-a quick and dirty usability scale," *Usability evaluation in industry*, vol. 189, no. 194, pp. 4–7, 1996.

[23] S. G. Hart and L. E. Staveland, "Development of nasa-tlx (task load index): Results of empirical and theoretical research," in *Advances in psychology*. Elsevier, 1988, vol. 52, pp. 139–183.

[24] Anysphere, "Cursor: Ai-powered code editor," https://cursor.com/, Anysphere, 2023, aI-assisted integrated development environment. Available at https://cursor.com/.

[25] D. Friedman and A. B. Dieng, "The vendi score: A diversity evaluation metric for machine learning," 2022.

[26] Y. Zhang, M. Li, D. Long *et al.*, "Qwen3 embedding: Advancing text embedding and reranking through foundation models," 2025.

[27] M. Oquab, T. Darcet, T. Moutakanni *et al.*, "Dinov2: Learning robust visual features without supervision," 2024.

[28] S. Sirigiri, N. S. de Lara, C. Agia, F. Shkurti, and F. Ramos, "Diversity you can actually measure: A fast, model-free diversity metric for robotics datasets," 2026.

[29] H. Fang, J. Duan, D. Clay *et al.*, "Molmoact2: Action reasoning models for real-world deployment," *arXiv preprint arXiv:2605.02881*, 2026.

[30] A. Deshpande, M. Guru, R. Hendrix *et al.*, "Molmob0t: Large-scale simulation enables zero-shot manipulation," *arXiv preprint arXiv:2603.16861*, 2026.

[31] P. Intelligence, K. Black, N. Brown *et al.*, "$\pi_{0.5}$: a vision-language-action model with open-world generalization," 2025.

[32] NVIDIA, :, J. Bjorck *et al.*, "Gr00t n1: An open foundation model for generalist humanoid robots," 2025.

[33] N. Agarwal, A. Ali, J. Allen *et al.*, "Cosmos 3: Omnimodal world models for physical ai," *arXiv preprint arXiv:2606.02800*, 2026.

[34] J. Zheng, J. Li, Z. Wang *et al.*, "X-vla: Soft-prompted transformer as scalable cross-embodiment vision-language-action model," in *International Conference on Learning Representations*, vol. 2026, 2026, pp. 60 580–60 606.

[35] J. Sun, W. Zhang, Z. Qi *et al.*, "Vla-jepa: Enhancing vision-language-action model with latent world model," in *European Conference on Computer Vision*. Springer, 2026, pp. 478–497.

[36] T. Yuan, Z. Dong, Y. Liu, and H. Zhao, "Fast-wam: Do world action models need test-time future imagination?" *arXiv preprint arXiv:2603.16666*, 2026.